3DG ENGINE

First Game Tutorial

COIN RUSH

A start-to-finish guide to scene building, Content scripts, collision, navigation, grounded AI, reactive Behavior Trees, target focus, animation, HUD, feedback, validation, and packaging.

Contents

Follow the milestones in order and verify each result before continuing.

Coin Rush is a compact third-person game that teaches the complete 3DG Engine workflow. The player explores an arena, collects every coin, avoids an enemy, and wins when the final coin is collected. The enemy uses navigation and a behavior graph to find, chase, and damage the player.

By the end you will have:

- a saved project scene with lighting, collision, clouds, and shadows;
- a capsule-based third-person player with health;
- non-blocking collectible coins using the Collectible collision preset;
- C++ gameplay scripts registered in the shared game module;
- a Nav Mesh Bounds Volume and visible navigation debug coverage;
- an enemy behavior graph with tasks, decorators, a service, and blackboard data;
- grounded enemy movement with gravity, floor probing, slopes, and steps;
- an Animation Graph driven by Speed, VerticalSpeed, and movement-state flags;
- a HUD for health, score, elapsed time, and game state;
- audio and particle feedback;
- a validated runtime scene and a packaged standalone game.

> *This guide matches the current engine workflow. Editor scene files belong under `Content/Scenes`, reusable assets belong under `Content/Assets`, F7 exports a runtime scene, and F8 validates it.*

---

1. Production roadmap

Milestone	Deliverable	Completion check
M0	Project and scene folders	The scene saves and reopens from `Content/Scenes`
M1	Graybox arena	The player cannot leave through the walls
M2	Player	Walk, run, jump, health, and camera collision work
M3	Coin prototype	A coin overlaps the player without blocking the player or camera
M4	Gameplay scripts	A collected coin adds score once and disappears
M5	Navigation	Navigation debug covers the intended walkable floor
M6	Enemy AI and movement	The enemy remains grounded, patrols, detects, chases, and attacks
M7	HUD and game rules	Health and score update; victory and defeat work
M8	Feedback	Pickup and damage are readable through sound and particles
M9	Runtime release	F8 passes and the packaged player launches the game

Build one milestone at a time. Do not duplicate ten coins until one coin works, and do not tune the enemy until the navigation preview is correct.

---

2. Prepare the project

2.1 Build the tools

From the repository root on Windows:

```powershell
cmake -S . -B build
cmake --build build --config Debug --target 3DGEditor player
```

Run the editor from:

```text
build/bin/editor/Debug/3DGEditor.exe
```

2.2 Create the content layout

Use the Content browser to create this structure:

```text
Content/
  Scenes/
  Assets/
    AI/
    Animations/
    Audio/
    HUD/
    Materials/
    Particles/
    Textures/
```

Create a new scene and save it as:

```text
Content/Scenes/CoinRush.scene
```

Use stable object names throughout the tutorial:

```text
PlayerStart
Enemy
GameManager
Coin_01, Coin_02, Coin_03 ...
```

Frequent saving matters. Press F5 after each milestone and keep the Console panel visible while testing.

---

3. Build the arena

3.1 Ground and boundary walls

1. Choose **Add > Primitives > Plane** and name it `Ground`.
2. Scale the ground to roughly `20, 1, 20`.
3. Enable its collider and choose the **World Static** collision preset.
4. Add four cubes and scale them into boundary walls.
5. Give every wall a Box collider with the **World Static** preset.
6. Add a staircase or a few cubes as obstacles, leaving wide routes for the enemy.

The collision preset is more important than the visual mesh. A visible wall without a collider will not block anything.

3.2 Lighting and sky

1. Add a Directional Light.
2. Open **World Settings** and set a readable time of day.

3. Enable Sun Shadows.
4. Enable **Skylight Occlusion (Indoor Darkness)** so enclosed spaces do not remain evenly bright. Tune Indoor Occlusion Strength and Minimum Skylight.
5. Optionally enable Clouds, choose a cloud preset, and enable Cast Cloud Shadows.

For the first playtest, favor clear lighting over dramatic darkness. You can tune the look after gameplay works.

3.3 Arena test

Save the scene. Add the player in the next section, then verify every wall before continuing. If the player passes through a wall, inspect the collider rather than the material or rendered mesh.

---

4. Add the player

1. Choose **Add > Gameplay > Player Start**.
2. Keep the name `PlayerStart`. Scripts and AI will use this name.
3. Place the capsule just above the ground, not inside it.
4. In Player Controller, use these starting values:

Setting	Suggested value
Walk Speed	4.0
Run Speed	7.0
Jump Speed	5.0
Capsule Radius	0.4
Capsule Height	1.8
Camera Distance	5.0
Camera Target Height	1.0
Max Slope	50 degrees
Step Height	0.35

1. Confirm that Health is enabled and set to `100 / 100`.
2. Leave the player on the **Player** collision channel. Player Start configures this automatically.

Press Play and test:

- WASD movement;
- Shift to sprint;
- Space to jump;
- mouse look;
- V to toggle first-person and third-person views;
- camera retraction against a wall;
- smooth camera return after leaving the wall.

The runtime uses a trigger-only player overlap capsule in addition to the movement controller. It can receive coin and camera-zone overlaps without physically fighting the character controller.

---

5. Create the coin prototype

5.1 Visual object

1. Add a Sphere, Cylinder, or Torus and name it `Coin_01`.
2. Scale it to approximately `0.35` to `0.45` units.
3. Give it a yellow or gold material. A small emissive value helps it remain visible.
4. Add a Rotator component for a slow Y-axis spin.

5.2 Correct collision setup

In the Collider section, choose:

```text
Collision Preset: Collectible
Object Channel:   Collectible
Overlap Only:     On
Responds To:      Player
```

The Collectible preset has two important effects:

- the player capsule passes through the coin while overlap events are still emitted;
- the third-person camera ignores the coin, so the camera does not retract or clip forward when a coin is between the player and the camera.

Do not use Default blocking collision for a pickup. If you need a completely non-interactive decoration, use **No Collision** instead.

5.3 Prototype test

Enter Play mode and walk through the coin before adding scripts. The player should not stop, slide, jump, or push the coin. The camera should also ignore it.

---

6. Write the coin script

Gameplay scripts are compiled into the shared `game` module. Both editor Play mode and the standalone player link this module, so an editor-created script is registered once and runs in both.

1. Select `Coin_01`.
2. Expand its Script component in the Inspector.
3. Enter `Coin` in **Script Class**.
4. Choose the **Pickup** template.
5. Click **Create Script**.

The editor now creates `Content/Scripts/Coin.h`, adds `Coin` to the shared generated registry, attaches it to the coin, enables it, and creates the `target`, `score`, and `particleBurst` fields. You do not need to locate or register the file manually.

Expand **Script Source** to view or change the generated C++ without leaving the editor. You can also open `Content/Scripts` in the Assets panel and double-click a script to open the standalone Script Editor window. Click **Save Source** after making changes.

This script uses `WasTriggerEntered` rather than a distance check. It therefore honors the collision system, fires once, and works with the player's overlap proxy.

6.1 Compile the native script

Scripts are native C++, so a newly created or edited class must be linked into the editor and player. In the Script Editor, click **Compile Scripts & Restart**. The editor saves the source and scene, closes so its executable is no longer locked, builds both `3DGEditor` and `player`, and reopens automatically. The Console reports whether compilation succeeded. Use **Last Build Log** in the Script Editor to inspect compiler errors.

You can keep the built-in source editor or choose **VS Code**, **Visual Studio**, **Rider**, or **Custom** from the **Code Editor** setting. Double-clicking scripts in `Content/Scripts` uses the saved preference. A custom editor receives the script path as its first argument.

6.2 Configure and test

1. Select `Coin_01`.
2. Confirm **Script Enabled** is checked and the class is `Coin`.
3. Keep `target` as `PlayerStart`, set `score` to `10`, and set `particleBurst` to `16`.
4. Enter Play mode and collect the coin.

The Console must not report an unregistered script. The score should increase once, and the coin should disappear.

---

7. Add the victory tracker

Create this one through the editor too:

1. Add an empty or simple object named `GameManager`.
2. Disable its collider if it does not need one.
3. In the Script component, enter `GoalTracker`, choose **Empty**, and click **Create Script**.
4. Expand **Script Source**, replace the generated class body with the example below, and click **Save Source**.

```cpp
#pragma once

#include <engine/ecs/Components.h>
#include <engine/gameplay/GameMode.h>
#include <engine/gameplay/Script.h>

class GoalTracker final : public engine::Script {
public:
    void OnUpdate(float) override {
        int remaining = 0;
        Registry()->view<engine::ecs::RuntimeName>().each(
            [&](engine::ecs::Entity, engine::ecs::RuntimeName& name) {
                if (name.value.rfind("Coin_", 0) == 0) ++remaining;
            });

        if (remaining > 0) sawCoin = true;
        if (sawCoin && remaining == 0 && engine::GameMode::Instance().IsPlaying()) {
            engine::GameMode::Instance().Win("All coins collected!");
        }
    }

private:
    bool sawCoin = false;
};
```

Then:

1. Save the scene, close the editor, and rebuild the editor and player.
2. Restart the editor and confirm `GoalTracker` remains attached and enabled.
3. Enter Play mode and collect every coin.

The built-in GameMode loses the run when the player Health reaches zero. GoalTracker adds the victory condition when no `Coin_` object remains.

---

8. Duplicate the collectible pass

Only after `Coin_01` works:

1. Duplicate it several times.
2. Rename the copies `Coin_02`, `Coin_03`, and so on.
3. Scatter them across the arena.
4. Keep them above the floor and away from unreachable ledges.
5. Confirm every duplicate retains the Collectible preset and Coin script.

The `Coin_` prefix is required by GoalTracker. A coin named `GoldPickup` will still add score, but GoalTracker will not count it.

---

9. Build navigation

9.1 Navigation bounds

1. Choose **Add > AI > Nav Mesh Bounds Volume**.
2. Scale and position it so it encloses the arena floor and all intended routes.
3. Do not extend it into areas the enemy should never reach.
4. Enable the navigation debug display from the Debug menu.

The colored navigation overlay should cover walkable ground, pass through door-sized gaps, and avoid walls and large obstacles.

9.2 Repair navigation before AI

If the overlay is missing or broken:

- enlarge or reposition the Nav Mesh Bounds Volume;
- verify the ground and walls have colliders;
- use World Static collision for level geometry;
- widen narrow gaps;
- remove accidental blocking colliders;
- rebuild or refresh the navigation preview.

Do not compensate for a broken nav mesh by increasing enemy speed.

---

10. Create the enemy

1. Add a Capsule and name it `Enemy`, or add an imported animated character and keep a capsule collider around it.
2. Give the placeholder a red material or assign the character's material.
3. Set its channel to **Enemy**.
4. Add Health, for example `50 / 50`.

5 Add a Nav Agent component.

6 Set Target to `PlayerStart`.

7 Use initial values such as:

Nav Agent setting	Suggested value
Speed	3.2
Max Force	25
Reach Radius	1.25
Repath	0.30 seconds
Vision Range	14
Vision Half-Angle	65 degrees
Team	2

Expand **AI Movement** and start with:

AI Movement setting	Suggested value
Movement Mode	Grounded / Falling
Gravity	-9.81 m/s2
Maximum Fall Speed	35 m/s
Ground Probe	0.25 m
Step Height	0.35 m
Maximum Slope	50 degrees

Grounded navigation only supplies horizontal intent. AI Movement owns vertical velocity, gravity, floor contact, slopes, and steps. This prevents a ground enemy from flying upward toward a target on another level. Use **Flying** only for an enemy that is intentionally allowed to move in three dimensions.

Reach Radius is the enemy's stopping and attack zone. Once the player's horizontal distance is at or below this value, the enemy clears its remaining movement velocity and holds position. It continues ticking the Behavior Tree, so Attack and Cooldown still work. If the player moves outside Reach Radius, Chase resumes automatically.

For a staircase, **Step Height** must be at least the height of one individual riser, not the total staircase height. The engine resolves a Staircase collider as separate treads, allowing the capsule to climb one step at a time. If a custom stair mesh is used, give it matching stepped collision or use simple blocking boxes for the treads.

Every floor that should support the enemy needs a solid collider whose response blocks the **Enemy** channel. Terrain is sampled directly. A rendered mesh without a blocking collider is not an AI floor.

If using automatic hostile acquisition, place the player and enemy on different non-zero teams. Otherwise the explicit `PlayerStart` target is enough.

Add at least two patrol points through the Nav Agent inspector by moving the enemy between point additions.

---

11. Build the behavior graph

Open **Panels > Behavior Graph**. Save the graph under:

```text
Content/Assets/AI/CoinRushEnemy.btgraph
```

11.1 Blackboard

Add this initial blackboard value:

Key	Type	Value	Purpose
aggressive	Bool	true	Enables the attack branch

The runtime also supplies target position and visibility through AgentContext. The authored blackboard is for persistent named game data shared by the tree.

11.2 Node layout

Build this tree:

```text
Selector (Root)
  Sequence: Visible Target
    See Target?
    Focus Target
    Selector: Combat
      Attack
        Decorator: Check Bool, key aggressive, value true
        Decorator: Target Within, range 1.25
        Decorator: Cooldown, 0.75 seconds
      Chase
        Service: Repath, 0.30 seconds
  Sequence: No Visible Target
    Clear Focus
    Patrol
```

How the parts map to behavior-tree concepts:

- **Tasks:** Focus Target, Clear Focus, Attack, Chase, and Patrol perform work.
- **Decorators:** Check Bool, Target Within, Cooldown, and Sees Target decide whether a branch may execute.
- **Service:** Repath updates the chase route while that branch is active.
- **Blackboard:** `aggressive` is persistent shared data used by a decorator.

`See Target?` uses the Nav Agent's Vision Range, Vision Half-Angle, and unobstructed line of sight. **Focus Target** succeeds only when that perception check is true and keeps the enemy facing the player, even while an attack montage locks movement. When visibility is lost, the reactive root selects the fallback sequence and **Clear Focus** returns facing control to locomotion before Patrol begins.

Select the Selector and choose Set as Root. Connect its children in the order shown. Set Attack Damage to a moderate value such as `8`.

11.3 Assign and debug

1. Save the graph.
2. Select `Enemy`, open Nav Agent, and choose `CoinRushEnemy.btgraph` from the **Behavior Tree** dropdown. Drag-and-drop is not required.
3. Enter Play mode.
4. Open the Behavior Graph panel and watch node borders: running, success, and failure states show live execution.

The Content browser recognizes `.btgraph` files as Behavior Tree assets. Double-clicking one opens it in the Behavior Graph panel. The panel's **Saved Trees** dropdown lists graphs from every Content subfolder. **Save** overwrites the graph currently open; it does not create a duplicate asset.

Expected result:

- far from the player, Patrol runs;
- outside sight, Clear Focus runs before Patrol;

- when the player is visible, Focus Target activates and Chase periodically repaths;
- within Reach Radius, movement stops completely while Attack and Cooldown continue;
- when the player leaves Reach Radius, Chase automatically resumes;
- when player Health reaches zero, GameMode enters Game Over.

Behavior Trees decide what the enemy wants to do. They do not own grounded, falling, or flying animation state; AI Movement publishes those values directly to the Animation Graph.

---

12. Configure enemy locomotion and airborne animation

Skip this section if the enemy is still a non-animated capsule. For an animated character, select `Enemy`, open **Panels > Animation Preview**, and create or edit the State Graph.

AI Movement automatically provides these Animation Graph parameters:

Parameter	Type	Meaning
Speed	Float	Horizontal movement speed
VerticalSpeed	Float	Current upward or downward velocity
IsGrounded	Bool	The ground probe found a walkable floor
IsFalling	Bool	Gravity is moving the enemy without floor support
IsFlying	Bool	AI Movement is intentionally in Flying mode

Create animation states such as:

```text
Idle
Walk
Run
Fall
Land
Attack
```

For a simple state graph, use transitions like:

From	To	Condition
Idle	Walk	Speed >= 0.15
Walk	Run	Speed >= 3.0
Run	Walk	Speed < 3.0
Walk	Idle	Speed < 0.15
Any State	Fall	IsFalling != false
Fall	Land	IsGrounded == true
Land	Idle	Exit time reached and Speed < 0.15

Use `VerticalSpeed < 0` when you need to distinguish falling from upward motion. Attack may be a state or a non-layered action depending on the character setup. A non-layered action blocks navigation movement until it completes, while a layered upper-body action can play without stopping locomotion.

Test the graph by letting the enemy walk over slopes and steps, then temporarily place it above the floor. It should enter Fall, land on the collider, play Land, and return to locomotion without rising toward the player's height.

---

13. Build the HUD

Open **Panels > HUD Editor**, choose New, and create these widgets:

Widget	Binding	Key or text	Suggested anchor
Bar	Health Fraction	Player health	Top Left
Text	Health Text	Health value	Top Left
Text	Named String	key `score`, text `Score: {}`	Top Right
Text	Named Float	key `time`, text `Time: {}`	Top Right
Text	Named String	key `gamestate`, text `{}`	Top Center
Text	Named String	key `gamemessage`, text `{}`	Center
Button	Restart Play action	Restart	Center or Bottom Center

Save the HUD as:

```text
Content/Assets/HUD/CoinRush.hud
```

Click **Use in Scene**. The runtime supplies `score`, `time`, `gamestate`, and `gamemessage`, while the health bindings use the player Health component.

Test at the target window size. Keep important widgets inside safe screen margins.

---

14. Add audio and particles

13.1 Coin feedback

On the coin prototype:

1. Add an Audio Source with a short pickup sound.
2. Disable looping and use the SFX bus.
3. Add a Particle System with a short burst preset.
4. Disable continuous emission if it should burst only on pickup.

The Coin script calls `PlayAudio(true)` and `BurstParticles(16)` before delayed destruction. Duplicate the fully configured prototype so every coin inherits the same feedback.

13.2 Damage feedback

Use a damage audio source, impact particles, or a small scripted camera shake. Keep feedback restrained enough that repeated AI attacks remain readable.

---

15. Full playtest checklist

Start from a fresh Play session and verify:

- PlayerStart appears above the ground.
- Movement, sprint, jump, and camera controls work.
- Walls and world objects block the player.

- Coins do not block the player.
- Coins do not retract the third-person camera.
- Each coin changes score exactly once.
- Pickup sound and particles play.
- Navigation debug covers every intended enemy route.
- The enemy patrols when the target is unavailable.
- The enemy chases and repaths when the target is visible.
- The enemy attacks only in range and respects cooldown.
- The enemy remains on collider floors, falls under gravity, and never follows the player's Y position like a ghost.
- The Animation Graph changes between locomotion, falling, and landing using the AI Movement parameters.
- Player Health updates on the HUD.
- Zero player Health produces Game Over.
- Collecting the last coin produces Victory.
- Restart begins with full health, zero score, and all coins restored.

Fix Console warnings before export. Missing scripts, assets, behavior graphs, or HUD files should never be ignored during release testing.

---

16. Save, export, and validate

1. Return to Edit mode.
2. Press F5 to save the editor scene.
3. Confirm the saved scene is under `Content/Scenes`.
4. Press F7 to export the runtime scene.
5. Press F8 to validate runtime content.

Runtime export includes the current collision layer and response masks. The Collectible and Player channel relationship therefore remains the same outside the editor.

Validation should report no missing materials, textures, scripts, HUD assets, behavior graphs, or particle/audio assets.

---

17. Configure and package the player

Point `player.cfg` at the exported scene. A typical configuration is:

```ini
window.width = 1280
window.height = 720
window.vsync = true
player.scene = game/Scenes/CoinRush.runtime.scene
```

Build and test the standalone player:

```powershell
cmake --build build --config Release --target player
```

For a staged package, configure `PLAYER_GAME_DIR` to the folder containing the runtime scene and required assets, then install the player component:

```powershell
cmake -S . -B build-release `
  -DPLAYER_GAME_DIR=D:/Games/CoinRush `
  -DGAMEENGINE_STATIC_RUNTIME=ON
cmake --build build-release --config Release --target player
cmake --install build-release --config Release --prefix dist --component player
```

Test the packaged executable from outside the source and build folders. This catches accidental absolute paths and missing copied assets.

---

18. Troubleshooting

The player stops on a coin

- Select the coin and set Collision Preset to Collectible.
- Confirm Overlap Only is enabled.
- Confirm the Object Channel is Collectible.
- Confirm the player uses Player Start or the Player channel.

The camera retracts when a coin is behind the player

- The coin is probably still on Default, World Static, or Camera Blocker.
- Apply Collectible and re-export the runtime scene.

The coin does not collect

- Confirm the Coin script is registered and Script Enabled is checked.
- Confirm the player is named `PlayerStart`.
- Confirm the coin responds to Player in Channel Responses.
- Confirm the player overlap proxy exists by testing another trigger or camera zone.
- Check the Console for an unregistered script message.

The enemy does not move

- Confirm navigation debug covers the enemy and player positions.
- Confirm the Nav Mesh Bounds Volume encloses the floor.
- Confirm the correct graph is selected in the Nav Agent **Behavior Tree** dropdown.
- Confirm the graph has a valid root and linked children.
- Confirm Target is `PlayerStart` and vision settings are not zero.

The enemy floats, moves upward, or falls through the floor

- Set AI Movement Mode to **Grounded / Falling**, not Flying.
- Confirm Gravity is negative and Maximum Fall Speed is greater than zero.
- Confirm the floor has a solid collider that blocks the Enemy channel.
- Increase Ground Probe slightly if the enemy flickers between grounded and falling.
- Check Step Height and Maximum Slope against the level geometry.

- Do not use Behavior Tree blackboard keys for `IsGrounded` or `IsFalling`; those are engine-provided Animation Graph parameters.

The enemy animation stays in locomotion while falling

- Open Animation Preview and confirm the State Graph has a Fall state.
- Use `IsFalling != false` for the transition into Fall.
- Use `IsGrounded == true` for Fall to Land.
- Confirm the animated model is on the same object as the Nav Agent.

The enemy reaches the player but does not damage health

- Confirm the player has Health.
- Increase Reach Radius slightly.
- Confirm Attack Damage is greater than zero.
- Check the Target Within decorator and Cooldown values.

The enemy keeps moving into the player

- Confirm Reach Radius is larger than the combined player and enemy collider radii.
- Use the same range for the Attack branch's Target Within decorator.
- Confirm Chase is the movement task; the engine clears its velocity inside Reach Radius.
- If the enemy should attack from farther away, increase Reach Radius instead of increasing the capsule size.

The enemy does not face the player or stays focused after losing sight

- Put **See Target?** before **Focus Target** in the visible-target sequence.
- Put **Clear Focus** first in the fallback sequence before Patrol or Idle.
- Confirm the enemy has a valid Chase Target and non-zero Vision Range.
- Increase Vision Half-Angle while testing if the player begins behind the enemy.
- Check that walls block visibility and trigger volumes do not.

The enemy jumps to the top of stairs or cannot climb them

- Use a Staircase collider or individual blocking tread colliders.
- Set Step Height to at least one riser height.
- Do not set Step Height to the total height of the staircase.
- Confirm every tread blocks the Enemy channel.
- Keep Maximum Slope for ramps; stair climbing is controlled primarily by Step Height.

The HUD does not update

- Click Use in Scene after saving the HUD.
- Check the exact lowercase keys: `score`, `time`, `gamestate`, `gamemessage`.
- Use Health Fraction or Health Text for player health.

An enclosed room remains too bright

- Enable Skylight Occlusion (Indoor Darkness).
- Raise Indoor Occlusion Strength.
- Lower Minimum Skylight carefully.

- Confirm light sources and shadow settings are appropriate for the room.

19. Completion gate

Coin Rush is complete when all of the following are true:

1. The editor scene reopens from `Content/Scenes` without errors.
2. A clean Play session can reach both Victory and Game Over.
3. Coins overlap rather than block and never interfere with the camera.
4. AI movement remains grounded on blocking floors and inside intended navigation.
5. The behavior graph focuses a visible player, clears focus after sight is lost, stops inside Reach Radius, and switches between Patrol, Chase, and Attack.
6. The Animation Graph responds to Speed, IsFalling, and IsGrounded.
7. HUD, audio, particles, score, and health update correctly.
8. F8 validation passes.
9. The standalone packaged executable launches without the editor.

You now have the complete first-game loop: build a scene, author gameplay, connect navigation and AI, drive a HUD, validate the runtime data, and ship a standalone game.